

LEAP: Supercharging LLMs for Formal Mathematics with Agentic Frameworks

Po-Nien Kung¹, Linfeng Song², Dawsen Hwang³, Jinsung Yoon¹, Chun-Liang Li¹, Simone Severini², Mirek Olšák³, Edward Lockhart³, Quoc V Le³, Burak Gokturk¹, Thang Luong³, Tomas Pfister¹ and Nanyun Peng¹

¹Google Cloud AI Research, ²Google Cloud, ³Google DeepMind

Large Language Models (LLMs) exhibit strong informal mathematical reasoning but struggle to generate mechanically verifiable proofs in formal languages like Lean. We present LEAP (LLM-in-Lean Environment Agentic Prover), an agentic framework that enables general-purpose foundation models to achieve state-of-the-art performance on automated formal theorem proving. LEAP leverages foundation model capabilities, such as informal reasoning, instruction following, and iterative self-refinement. By decomposing complex problems into smaller units, the system bridges formal proof construction with informal blueprints through continuous interaction with the Lean compiler. To provide a rigorous evaluation beyond increasingly saturated benchmarks, we introduce Lean-IMO-Bench, a benchmark of IMO-style problems formalized in Lean, with short statements yet highly non-routine and multi-step proofs across a wide range of difficulty levels. Empirically, on the latest 2025 Putnam Competition, an annual mathematics competition for undergraduate students in North America, LEAP solves all 12 problems, matching recent breakthroughs by frontier formal mathematical models; On Lean-IMO-Bench, LEAP boosts the one-shot formal solve rate of general-purpose LLMs from below 10% to 70%, notably surpassing the 48% benchmark set by a specialized, gold-medal-caliber IMO system. Furthermore, we demonstrate LEAP’s research-level utility by autonomously formalizing complex proofs for open combinatorial challenges, including a verified proof for a key subproblem in Knuth’s Hamiltonian decomposition of even-order Cayley graphs.

1. Introduction

Large Language Models (LLMs) have made impressive progress on mathematical reasoning with natural language, also known as “informal math reasoning”, demonstrating strong capabilities in complex reasoning and problem-solving for both math competitions and research level maths (Feng et al., 2026a,b,c; Huang and Yang, 2025; Luong and Lockhart, 2025). However, as discussed in recent works like Hilbert (Varambally et al., 2025) and Goedel-Prover-V2 (Lin et al., 2026), solutions in natural language frequently suffer from logical fallacies and hallucinations, and they are hard to automatically verify. This difficulty in verification is not merely a limitation of automated systems; even for human mathematicians, verifying complex proofs is a notoriously time-consuming process requiring scarce expert labor (Greiffenhagen, 2024). A famous example is the proof of the Kepler conjecture (Hales, 2005), which required four years of peer review before the referees could only claim to be “99% certain” of its correctness,¹ eventually necessitating a decade-long formal verification effort (Hales et al., 2017). This verification bottleneck underscores that assessing correctness is a hard task in natural language, motivating the exploration of formal mathematics, where proofs are written in a machine-checkable language and verified by a rigorous kernel like in Lean (Moura and Ullrich, 2021), Isabelle (Nipkow et al., 2002), Coq (Huet et al., 1997), HOL Light (Harrison, 2009), offers automated verification with guaranteed accuracy. Yet, bridging the gap to formal theorem proving remains a significant challenge, and the performance of automated formal provers currently lags substantially behind that of general-purpose LLMs operating in natural language.

¹https://en.wikipedia.org/wiki/Kepler_conjecture

To bridge this gap, recent efforts in the research community predominantly fine-tunes specialized prover models (e.g., AlphaProof (Hubert et al., 2026), DeepSeek Prover V2 (Ren et al., 2025), Seed Prover (Chen et al., 2025b), Goedel Prover V2 (Lin et al., 2026)) on formal corpora, with the assumption that general LLMs are ineffective for rigorous formal tasks without specialization. Indeed, according to the FormalProofBench (Ravi et al., 2026)² and TaoBench (Taylor et al., 2026), general LLMs often underperform compared to specialized prover models.

While some recent works explored agentic or inference-time search, they still depend on specialized models. For instance, Hilbert (Varambally et al., 2025), AlphaProofNexus (Tsoukalas et al., 2026), Aristotle (Achim et al., 2025), and Seed Prover V1.5 (Chen et al., 2025a) use general LLMs for informal reasoning but rely on specialized models for Lean proving steps. Axiom³ and Numina⁴ claim strong results on Putnam 2025 while remained closed source without public access, making them scientifically unverifiable.

In this paper, we show that while general LLMs remain limited in one-shot theorem proving, the bottleneck is not language comprehension but generating long, complex, correct proofs in one attempt. General LLMs offer complementary skills to specialized models: strong informal reasoning, instruction following, tool use, and self-refinement. These make them ideal for agentic ATP frameworks, where proof construction is decomposed and iteratively improved. To this end, we introduce **LEAP (LLM-in-Lean Environment Agentic Prover)**, an agentic framework using *only* general LLMs for formal math. Inspired by human workflow, LEAP generates a high-level blueprint forming a directed acyclic graph (DAG), then generates the Lean proof, iteratively correcting errors via compiler feedback.

To evaluate progress beyond saturated benchmarks such as MiniF2F (Zheng et al., 2022) and PutnamBench (Tsoukalas et al., 2024), we introduce Lean-IMO-Bench, formalizing the challenging informal math benchmark IMO-Bench (Luong et al., 2025) problem statements into Lean. In contrast to existing benchmarks, which either focus on shorter problems or emphasize broad undergraduate coverage, Lean-IMO-Bench targets the complementary regime of elementary statements whose solutions often hinge on highly non-routine insights and unfold through long, multi-step, and structurally intricate proofs, providing a sharper test of formal theorem proving.

Empirically, on the latest 2025 Putnam Competition, a challenging annual undergraduate mathematics competition in North America whose 2025 top score was 110 out of 120 while the median was only 2, LEAP solves all 12 problems in Lean, achieving perfect performance. This matches recent breakthrough results from frontier formal mathematical reasoning models such as Axiom³ and Numina⁴. On Lean-IMO-Bench, LEAP substantially improves general LLMs’ solve rate from under 10% to 70%, surpassing specialized ATP models (5%) and Aristotle (48%), a strong system with specialized ATP components that earned the score for Gold medal at IMO 2025. The contribution of the paper is three-fold:

- **Workflow-Inspired Agentic Design** We introduce LEAP, an agentic framework that codifies the human mathematical workflow – combining high-level blueprint sketching with low-level formal proof generation and iterative compiler feedback. Crucially, LEAP demonstrates that state-of-the-art formal theorem proving can be achieved using *only* general-purpose LLMs, challenging the belief that specialized fine-tuning is indispensable.
- **Lean-IMO-Bench Dataset:** To evaluate progress beyond saturated benchmarks such as MiniF2F and PutnamBench, we introduce Lean-IMO-Bench, a new challenging dataset that translates

²The paper is associated with a private dataset with a live leaderboard https://www.vals.ai/benchmarks/proof_bench. We contacted them several times to participate on the leaderboard without receiving a response.

³<https://github.com/AxiomMath/putnam2025>

⁴<https://github.com/project-numina/Numina-Putnam2025>

informal problem statements from IMO-Bench into formal Lean statements. Resources are available at https://github.com/google-deepmind/superhuman/blob/main/imobench/lean_proof_bench.csv.

- **State-of-the-Art Results and Insights:** LEAP solves all 12 problems on Putnam 2025, and outperforms state-of-the-art baselines on Lean-IMO-Bench by a large margin. Our analysis yields the insight that the primary bottleneck in formal math for general LLMs is not formal language comprehension, but the lack of structured, iterative interaction with the proof environment.

2. LEAP: Blueprint-Driven Automated Theorem Proving

2.1. Formalizing Proofs with Blueprints

Formalizing mathematical proofs is rarely a one-shot task: it requires a structured plan for progressively translating a high-level argument into Lean. To manage this complexity, recent formalization efforts often use the Lean Blueprint tool⁵, which let mathematicians write a human-readable proof roadmap linked to Lean code and visualized as a directed acyclic graph (DAG), where each node represents a proof obligation. This workflow has been instrumental in coordinating large-scale projects such as the formalization roadmap for Fermat’s Last Theorem⁶, where a multi-year proof effort is organized through an explicit dependency graph.

Inspired by this workflow, we introduce LEAP, an agent for automated theorem proving with hierarchical decomposition and planning. Rather than synthesizing a complete proof in a single pass, LEAP incrementally drafts blueprints, decomposes Lean goals into supporting lemmas, and maintains the evolving proof plan as an AND-OR DAG.

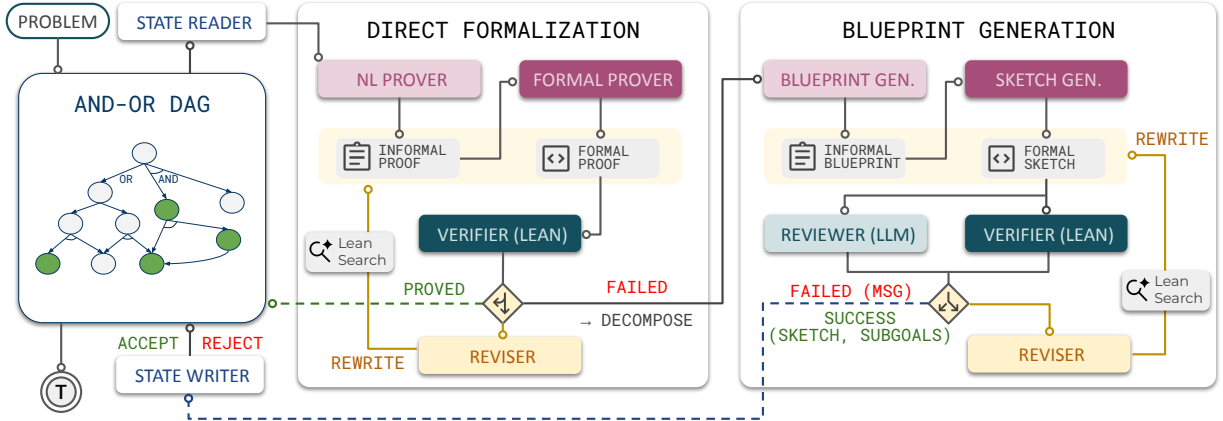


Figure 1 | **LEAP workflow.** LEAP first attempts direct formalization with compiler-feedback revision and LeanSearch(Gao et al., 2024) retrieval. If this fails, it generates an informal blueprint and formal proof sketch, adding verified subgoals back to the DAG only when dependencies remain acyclic.

2.2. Overview

Figure 1 illustrates the workflow of LEAP. Given an input theorem, LEAP registers its Lean statement as the root goal,⁷ represented as an OR node in the AND-OR DAG. To process an open goal, a *state*

⁵<https://github.com/PatrickMassot/leanblueprint>

⁶<https://leanprover-community.github.io/blog/posts/FLT-announcement/>

⁷A goal is any theorem or lemma statement to be proved; decomposition introduces *subgoals*. See Appendix B.

reader retrieves its statement, dependencies, and related lemmas. LEAP then first attempts a **direct proof** by generating an informal proof, translating it into Lean code, and checking the candidate with the Lean compiler.

If direct proving fails, LEAP shifts to **decomposition**. It first drafts an informal blueprint that proposes intermediate lemmas, then translates the blueprint into a Lean proof sketch. The sketch proves the current goal assuming only the proposed lemmas: the main theorem body is *sorry*-free, while *sorry* placeholders are permitted only in the newly proposed lemma statements. If the sketch is accepted by the Lean compiler, it is added as an AND node, and the proposed lemmas are added as child OR nodes. This ensures that once all child subgoals are proved, the parent goal is also proved. The verified sketch is then passed to the *state writer*, which checks that the update preserves acyclicity before committing it to the DAG. The agent then recursively processes the newly created subgoals.

This workflow relies on three tightly coupled design choices: **DAG-based hierarchical memoization**, which preserves progress and reuses lemmas across branches; **interleaved informal-formal planning**, which connects natural-language strategies with executable Lean code; and **verification-guided proof search**, which uses compiler feedback and LLM-based review to accept, revise, decompose, or abandon candidate branches.

2.3. Hierarchical Memoization via DAG

LEAP uses the AND-OR DAG not only to record proof progress, but also to structure hierarchical memoization. OR nodes represent open goals or lemma statements, each of which may be resolved by any valid proof strategy, while AND nodes represent candidate decompositions whose success depends on proving all constituent subgoals. [Figure 2](#) illustrates this structure.

The DAG provides two central advantages. First, **monotone refinement**: once a goal is decomposed into supporting subgoals, subsequent search can focus on expanding and resolving these descendants without restructuring the established dependency order. This separates local proof exploration from global proof organization: individual proof attempts may be revised, expanded, or abandoned, while the DAG preserves the stable dependency structure of the overall proof plan. Second, **lemma memoization**: intermediate lemma statements are stored as shared proof nodes and can be reused whenever the same subproblem arises in different branches. This also supports *anticipatory lemma planning*: during blueprint generation, LEAP may propose auxiliary lemma statements that are not immediately required by the current sketch but could support later proof steps. Such prospective lemmas remain available in the graph memory without being necessary for resolving the current AND node. Together, these properties allow independent proof plans to converge on common dependencies while reducing redundant derivations.

The resulting dependency structure also improves transparency: it exposes which goals remain open, which lemmas have been resolved, and which nodes block downstream progress. This helps LEAP identify where additional lemmas, revised decompositions, or stronger assumptions may be needed, while providing an interpretable blueprint-style workspace for human-AI collaboration.

2.4. Interleaved Informal–Formal Planning

As shown in [Figure 1](#), both the direct proof path and the blueprint decomposition path in LEAP pass through an informal proof sketch. This reflects the complementary strengths of LLMs and Lean: LLMs are effective at informal reasoning, strategy generation, and refinement, while Lean provides strict machine-checkable verification.

In direct proving, LEAP first generates an informal argument for the current goal before translating

it into a candidate Lean proof. In decomposition, it drafts an informal blueprint explaining how the goal can be reduced to supporting subgoals, then converts this plan into a Lean sketch that records the proposed dependencies. In both cases, the informal sketch provides a planning space before formalization, making proof construction less brittle than direct code generation alone (see [Appendix B](#) for examples of informal proofs and blueprints).

This interleaving also makes proof progress more interpretable: each formal attempt is paired with an informal rationale, allowing users to inspect why a proof step or decomposition was proposed rather than only reading Lean code or compiler feedback.

2.5. Verification-Guided Proof Search

As shown in [Figure 1](#), LEAP uses verification at two levels. First, the Lean compiler formally checks candidate proofs and sketches, ensuring that accepted code is syntactically valid and type-correct. For proof sketches, LEAP only permits `sorry` placeholders for the proposed subgoals (lemmas). This preserves the AND-OR semantics of the proof DAG: once all referenced subgoals are proved, the parent goal is also proved. Second, after a blueprint proposes new subgoals, an LLM reviewer assesses the quality of the decomposition: whether the subgoals are relevant to the parent goal, make the problem easier, and offer a plausible route to completing the proof. This planning-level review is crucial for complex goals, where a Lean sketch can be syntactically valid while introducing subgoals that are ill-posed or no simpler than the original statement. Without this filter, the agent may repeatedly expand weak blueprints, spending search budget on branches that do not make real progress. We study this failure mode through an ablation without the LLM reviewer in [subsection 5.3](#).

The LLM reviewer therefore acts as a search filter: it identifies unpromising decompositions, triggers backtracking, and encourages exploration of alternative strategies. Currently, LEAP uses a simple DFS over the DAG with backtracking. The effectiveness of this reviewer suggests a broader future direction: LLMs may also serve as heuristic evaluators for guiding search in formal proof spaces.

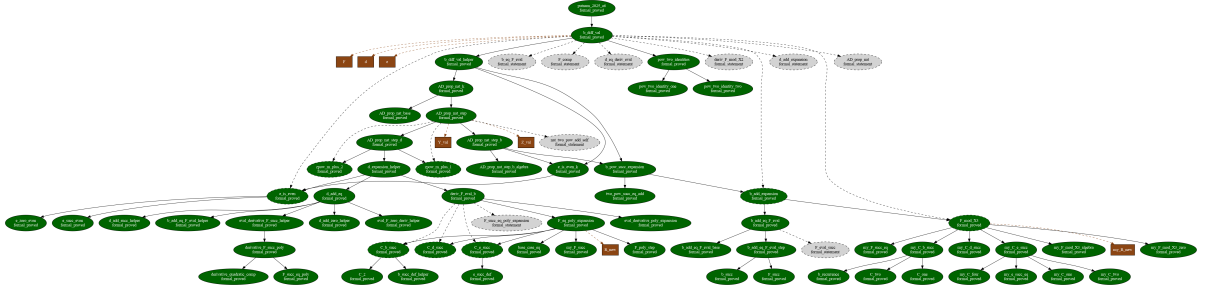


Figure 2 | **DAG example for Putnam 2025 Problem A6.** LEAP decomposes the theorem into a proof sketch and supporting lemmas. Through **anticipatory lemma planning**, the agent may also propose auxiliary lemma statements that are not immediately required but could be useful later; these are shown with dashed edges and are not needed to prove the main theorem. Green nodes are proven nodes, and brown blocks denote definitions, structures, or variables introduced at a node.

3. Lean-IMO-Bench: Formalizing IMO Problems in Lean

3.1. Lean-IMO-Bench

We introduce `LEAN-IMO-BENCH`, a curated collection of 60 problems building upon the foundational work of [Luong et al. \(2025\)](#). [Luong et al. \(2025\)](#) introduced `IMO-PROOFBENCH`, a rigorous suite

Table 1 | Baseline performance on LEAN-IMO-BENCH across three evaluation tasks. Natural Language Proof performance is based on human expert review.

Task	Model (Metric)	Basic Set (%)	Advanced Set (%)
Natural Language Proof	Gemini 2.5 Pro (Pass@1)	55.2	17.6
Formal Theorem Proving	Gemini 3.1 Pro (Pass@128)	20.0	3.3
	Gemini 3.1 Pro (Avg.@128)	4.6	0.2
Formal Proof Translation	Gemini 3.1 Pro (Pass@128)	20.0	3.3
	Gemini 3.1 Pro (Avg.@128)	4.6	0.8

vetted by an expert panel of mathematicians and IMO medalists. The benchmark contains 60 problems split evenly into a *Basic* set and an *Advanced* set of 30 problems each. The *Basic* set spans pre-IMO to IMO-Medium difficulty and includes 8 algebra, 8 combinatorics, 8 number theory, and 6 geometry problems. The *Advanced* set includes novel problems up to IMO-Hard difficulty, with 8 algebra, 8 combinatorics, 6 number theory, and 8 geometry problems. Overall, the benchmark is approximately balanced across algebra, combinatorics, geometry, and number theory.

To ensure the highest level of accuracy in LEAN-IMO-BENCH, Lean experts manually formalized and verified all 60 problem statements. Because these problems are at the IMO level, the required mathematical background is elementary. Consequently, we expect the corresponding Lean solutions to be concise, deliberately removing the overhead associated with formalizing complex, modern mathematical theories.

The dataset can be used to evaluate models across three distinct tasks: **Natural Language Proof**, **Formal Theorem Proving**, and **Formal Proof Translation**, while we focus on formal theorem proving in this paper. The baseline performance on LEAN-IMO-BENCH is summarized in Table 1. For the Natural Language Proof task, we cite Luong et al. (2025) as reference: Gemini 2.5 Pro shows strong informal reasoning performance. However, as shown in Table 1, this does not directly translate to formal theorem proving: Gemini 3.1 Pro performs substantially worse on Formal Theorem Proving, especially on the Advanced set. Providing a correct informal proof in the Formal Proof Translation task also yields little improvement, with Pass@128 unchanged and only a marginal gain in Average@128.

Table 1 demonstrates a stark gap in the models’ Lean capabilities. Because the model can already successfully solve these problems in natural language, mathematical reasoning is not the bottleneck, thus reliably generating valid Lean code remains the primary challenge.

4. Experimental Results

We evaluate LEAP with Gemini-3.1-pro as the backend large language model and compare it against four baselines: **Gemini-3.1-pro**, which tests one-shot proof generation by a strong general-purpose model; **Goedel-Prover-V2-32B** (Lin et al., 2026), a state-of-the-art open-source ATP model specialized for Lean; **Hilbert** (Varambally et al., 2025), an agentic Lean formalization framework that combines Goedel-Prover-V2-32B with Gemini-3.1-pro; and **Aristotle** (Achim et al., 2025), a specialized automated theorem-proving system with dedicated ATP components that achieved gold-medal-level performance at the 2025 IMO.

We evaluate formal proving ability on two datasets: **Putnam 2025** and our proposed **Lean-IMO-Bench**. Putnam 2025 contains twelve undergraduate-level problems from the 86th William Lowell

Putnam Mathematical Competition,⁸ a highly challenging North American mathematics competition. In the 2025 competition, the top score was 110 out of 120, the average score was approximately 10, and the median score was 2.

4.1. Results on Putnam 2025

Table 2 presents the evaluation results on the Putnam 2025 benchmark. Under a Pass@128 setting, the direct formalization baselines (Gemini-3.1-pro and Goedel-Prover-V2-32B) fail to solve any problems, indicating that single-pass generation is insufficient for the logical complexity of this dataset.

Table 2 | Putnam 2025 results. Green checkmarks (✓) indicate successfully solved problems, while red crosses (✗) indicate failures. Evaluation settings: [◊] indicates pass@128, while [†] indicates rollout=2.

Method	a1	a2	a3	a4	a5	a6	b1	b2	b3	b4	b5	b6	Solve Rate (%)
Gemini-3.1-pro [◊]	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	0.0
Goedel-Prover-V2-32B [◊]	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	0.0
Hilbert [†]	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	33.3
Aristotle [†]	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✗	✗	75.0
LEAP [†]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	100.0

The open-source agentic framework Hilbert improves upon direct generation, solving 4 out of 12 problems. However, during evaluation, we observed that Hilbert’s recursive search design leads to an exponential time complexity of $O((n \cdot b)^d)$, where n is the number of lemma retries, b is the average branching factor, and $d = 10$ is the maximum proof depth. Due to the high volume of redundant LLM calls required by this approach, we bounded each Hilbert rollout to a 7-day time limit. For context against state-of-the-art proprietary systems, we also report the performance of Aristotle. While the system is closed-source, it serves as a strong baseline, solving 9 out of 12 problems given two rollouts.⁹

LEAP successfully solves all 12 Putnam 2025 problems, improving the benchmark solve rate from 0% via direct formalization to 100% with our agentic framework. This performance directly results from the LEAP’s blueprint-inspired AND-OR DAG architecture, which resolves the search bottlenecks observed in standard recursive frameworks like Hilbert. By supporting hierarchical memoization, LEAP allows independent proof branches to reuse shared intermediate lemmas, significantly mitigating exponential search complexity and allow LEAP to solve problems efficiently. For a detailed, problem-level breakdown of the computational cost and search efficiency required to achieve these results, See Table 3 for runtime and efficiency statistics.

Table 3 | **Runtime and search efficiency of LEAP on Putnam 2025.** For each problem, we report the computational cost (total LLM calls for a verified proof), the search space explored (active DAG nodes/lemmas), and the final Lean proof line count.

Metric	a1	a2	a3	a4	a5	a6	b1	b2	b3	b4	b5	b6
LLM Calls	71	304	963	1.4k	3.0k	1.0k	116	46	98	87	239	161
Active Nodes	8	26	55	105	170	62	12	8	10	9	211	24
Proof Length	405	591	713	1.0k	2.0k	581	752	300	326	556	1.9k	848

⁸Mathematical Association of America, [Results of the 86th William Lowell Putnam Mathematical Competition](#).

⁹An [unofficial report](#) indicates Aristotle solved 10 out of 12 problems on this benchmark; however, neither that reported run nor our evaluation successfully solved problem A5.

4.2. Results on Lean-IMO-Bench

Table 4 presents evaluation results on Lean-IMO-Bench. We include this dataset to test model robustness across a broader spectrum of mathematical disciplines and distinct complexity tiers, providing a complementary challenge to the Putnam benchmark.

Direct formalization baselines (Gemini-3.1-Pro and Goedel-Prover-V2-32B) and the open-source Hilbert framework struggle significantly on this dataset, exhibiting severe performance degradation on the Advanced set. While the proprietary Aristotle system resolves a majority of the Basic problems, its effectiveness drops sharply as complexity increases. Notably, across all evaluated methods, performance in the Geometry category remains near zero. This aligns with the well-established difficulty of formalizing olympiad-level geometry in Lean without the aid of supplementary, domain-specific frameworks. We retain this category strictly to evaluate general-purpose reasoning under extreme formalization constraints.

Against these baselines, LEAP achieves the highest overall solve rates, scoring 83.3% on the Basic set and 56.7% on the Advanced set. By effectively leveraging its DAG-based architecture, LEAP demonstrates strong domain generalization, maintaining a 100% solve rate in both Algebra and Number Theory regardless of the difficulty tier.

Table 4 | Results on LEAN-IMO-BENCH. We report the solve rate (%) across different mathematical categories for the **Basic** and **Advanced** sets separately. Evaluation settings: [◊] indicates pass@128, while [†] indicates rollout=2. Best results in each section are in bold.

Method	Algebra	Comb.	Num. Theory	Geometry	Overall
Basic Set (%)					
Gemini-3.1-Pro [◊]	37.5	12.5	25.0	0	20.0
Goedel-V2-32B [◊]	37.5	0	0	0	10.0
Hilbert [†]	62.5	25	50	0	36.6
Aristotle [†]	75	100	100	16.7	76.7
LEAP [†]	100	100	100	16.7	83.3
Advanced Set (%)					
Gemini-3.1-Pro [◊]	0	12.5	0	0	3.3
Goedel-V2-32B [◊]	0	0	0	0	0
Hilbert [†]	12.5	0	16.6	0	6.6
Aristotle [†]	37.5	12.5	33.3	0	20.0
LEAP [†]	100	25	100	12.5	56.7

5. Discussion

5.1. Beyond One-Shot Formalization

A central motivation of LEAP is that general foundation models can be effective iterative formalizers, even when they are not specialized Lean provers. While specialized provers are trained for formal proof synthesis, general models offer complementary capabilities such as instruction following, long-context reasoning, informal planning, tool use, and feedback-based revision.

To isolate this effect, we evaluate the *Direct Formalization* component labeled in Figure 1 under two settings. In the one-shot setting, each model is evaluated with Pass@128 over independently sampled proof attempts. In the iterative setting, each model receives a single initial attempt and up to 20 compiler-feedback revision steps, yielding a Pass@1 result under a smaller sampling budget. As shown

in Table 5, Goedel-Prover-V2-32B does not benefit from this feedback loop, while Gemini-3.1-pro improves substantially from 20.0% to 36.6%.

This suggests that iterative formalization depends on capabilities beyond local Lean proof synthesis. Interpreting compiler errors, maintaining context, and revising proof attempts over multiple steps can be as important as one-shot formal proving accuracy. These results support using a general foundation model as the reasoning backbone of LEAP, while leaving open the possibility of combining it with specialized provers for local proof generation.

Table 5 | **One-shot vs. iterative formalization** performance on Lean-IMO-Bench Basic set.

Model	One-shot (Pass@128)	Iterative (Pass@1)
Goedel-Prover-V2-32B	10.0	6.6
Gemini-3.1-Pro	20.0	36.6

5.2. Effect of DAG-Based Memoization

LEAP maintains proof progress as a DAG-based memory rather than an isolated decomposition tree. This allows intermediate lemmas to be stored as shared nodes and reused across branches, while exposing graph context such as existing goals, dependencies, and previously proposed lemmas.

To evaluate this design, we compare LEAP with a tree-structured variant that follows the same workflow but removes global lemma sharing. As shown in Table 6, the tree variant already substantially outperforms Hilbert (Varambally et al., 2025), which achieves 36.6% and 6.6% on the Basic and Advanced sets, respectively (Table 4). This indicates that interleaved informal-formal planning and verification-guided search are effective even without DAG-based memoization. The full DAG version further improves performance from 73.3% to 83.3% on the Basic set and from 40.0% to 56.7% on the Advanced set, showing the benefit of global proof memory.

The improvement is especially pronounced on harder categories, such as Advanced Algebra and Advanced Number Theory, where shared lemmas and graph context are more likely to matter. We attribute this gain to two effects. First, the DAG supports anticipatory lemma planning: auxiliary lemmas proposed at higher-level nodes can later be reused by downstream subgoals (Figure 2). Second, repeated subproblems can be shared across branches, avoiding the need to rediscover or reprove the same lemma multiple times. Together, these properties reduce redundant derivations and improve proof search efficiency.

Table 6 | **DAG memoization ablation.** Solve rate (%) by category on Lean-IMO-Bench Basic (B)/Advanced (A) sets.

Config.	Alg.		Comb.		NT		Geo.		Overall	
	B	A	B	A	B	A	B	A	B	A
w/o DAG (Naive Tree)	100	75	75	25	100	66.6	0	0	73.3	40.0
Full DAG	100	100	100	25	100	100	16.7	12.5	83.3	56.7

5.3. Toward LLM-Guided Proof Search

Compiler verification checks whether a proof sketch is formally well-typed, but not whether its decomposition is useful. A sketch may prove the parent goal from proposed lemmas that are unhelpful, overly difficult, or nearly equivalent to the original goal. In LEAP, the LLM reviewer acts as a local

```

-- Grandparent goal
theorem perms_a_equiv_b_card
  (m : ℕ) (s : Fin (m + 1) → ℤ×) (k : Fin (m + 2)) :
  Nat.card (PermsA m s k) = Nat.card (PermsB m s k) := by
  rw [← card_perms_a_eq m s k, ← card_perms_b_eq m s k]
  exact valid_perms_end_eq_sigma_card m s k

-- Intermediate lemma (unfolding the definitions)
lemma valid_perms_end_eq_sigma_card
  (m : ℕ) (s : Fin (m + 1) → ℤ×) (k : Fin (m + 2)) :
  (ValidPermsEnd (m + 1) s k).card = (Finset.sigma ...).card := by
  rw [card_perms_a_eq m s k, card_perms_b_eq m s k]
  exact card_perms_a_eq_card_perms_b m s k

-- Proposed subgoal (syntactically identical to the grandparent goal)
lemma card_perms_a_eq_card_perms_b
  (m : ℕ) (s : Fin (m + 1) → ℤ×) (k : Fin (m + 2)) :
  Nat.card (PermsA m s k) = Nat.card (PermsB m s k) := by
  sorry

```

Figure 3 | **Unproductive decomposition without LLM review.** The proposed subgoal restates the grandparent goal, so the decomposition is formally admissible but does not simplify proof search.

search heuristic: it filters candidate decompositions by judging whether they meaningfully simplify the parent goal before they are committed to the DAG.

We focus this ablation on Putnam 2025 Problem A5 because it is one of the most challenging cases in our evaluation, requiring the longest runtime and two rollouts for LEAP to formalize the proof successfully. Removing the LLM-based decomposition reviewer causes the agent to fail even after eight rollout attempts. This contrast suggests that local LLM review provides a useful search signal: it rejects weak decompositions early, triggers backtracking, and prevents the agent from spending rollouts on branches that do not make substantive progress. We further inspect decomposition traces from the ablated setting; a representative failure case is shown in Figure 3.

The decomposition in Figure 3 is formally admissible but does not simplify the mathematical state. The agent first unfolds the definitions in the grandparent goal to create an intermediate lemma, then folds them back into a proposed subgoal that is syntactically identical to the original statement. Without semantic review, this duplicate lemma is treated as a new step, causing the agent to repeat the same unproductive decomposition until its search budget is exhausted. This failure highlights the potential of LLM-guided proof search: a reviewer can assess whether a proposed lemma actually advances the proof, prune cyclic or non-simplifying branches, and direct compute toward more promising paths.

5.4. Perspective: General LLMs as Formal Provers: From Zero to Hero

As demonstrated by LEAP, the seemingly insurmountable gap between the poor one-shot theorem proving performance of general LLMs and state-of-the-art results can be effectively bridged by a well-designed agentic framework. By shifting the paradigm away from relying solely on small specialized LLMs, we show that the extensive knowledge, instruction following, and self-correction capabilities of foundation models are more than sufficient. When scaffolded correctly, these foundation models can progress from near zero formal math performance to solving highly complex problems.

While small specialized LLMs lack the overarching agentic capabilities of their foundation coun-

terparts, we acknowledge that they still hold value. A hybrid architecture combining the high-level, structural reasoning of a foundation model with the focused, formal step generation of a fine-tuned specialized model could be a highly effective design pattern. However, exploring this hybrid approach remains outside the scope of this paper, as our primary goal is to highlight the standalone power of general purpose LLMs in an agentic workflow.

6. Case Studies: Formalizing Open Problems in Combinatorics

Hamiltonian Decomposition of Directed Cayley Graphs. To evaluate LEAP on a highly complex mathematical task, we targeted a recently solved open problem in combinatorics: the Hamiltonian decomposition of the directed Cayley graph $\Gamma_m = \text{Cay}(\mathbb{Z}_m^3, \{e_1, e_2, e_3\})$ for even m . Originally posed by Donald Knuth, the problem asks whether the graph’s directed arcs can be partitioned into exactly three distinct, spanning Hamiltonian cycles. The informal mathematical proof for the even-case construction is exceptionally intricate, relying on heavy combinatorial analysis and localized defect routing across different layers of the graph. We focused our formalization efforts on a critical subproblem: rigorously verifying that the 2D planar projection of a single color class’s routing dynamics forms an unbroken mathematical cycle of length m^2 . The informal arguments for this specific dynamic span roughly 20 pages of dense piecewise maps, parity-dependent intervals, and complex cross-row transitions. To tackle a formal verification of this magnitude, we deployed LEAP, which successfully decomposed the monolithic informal proof into a granular, highly structured proof graph. By autonomously and systematically resolving the interdependent nodes of this graph, LEAP managed to fully verify the complex cycle-merging dynamics, ultimately synthesizing over 5000 lines of rigorous Lean 4 code to complete the formal proof for this subproblem.

Formalizing Erdős Problem 457. We further tested LEAP on Erdős Problem 457, a classic graph theory problem concerning the density of triangle-free graphs. Although this problem is already resolved, it served as an ideal benchmark to assess LEAP’s ability to autonomously reconstruct and verify established mathematical results. Tasked with deriving the known proof from first principles in Lean 4, LEAP effectively navigated the combinatorial constraints to confirm the theorem’s validity. This successful reproduction demonstrates LEAP’s capability to reliably translate complex, existing literature into high-assurance formal proofs without human intervention.

7. Conclusion and Future Work

The success of LEAP underscores a broader implication: modern general LLMs already possess the strong reasoning capabilities required to tackle rigorous, domain specific challenges, provided they are embedded in an appropriate structural design. In the realm of formal mathematics, this naturally takes the form of logical problem decomposition. By utilizing the language model’s reasoning ability to break down complex theorems into smaller, manageable sub-problems, and strictly verifying these steps via the Lean compiler, we create a robust pathway to mathematical truth.

Moving forward, a critical challenge will be optimizing how we navigate the resulting proof trees. As problems are decomposed into increasingly intricate sub-tasks, the search space grows rapidly. Future work must focus on how to efficiently search through this space, exploring methods to guide the agent’s focus and manage the complexity of large scale proof generation. Mastering this efficient exploration will be the key to unlocking even more complex mathematical milestones.

Acknowledgements

We thank Michael P. Brenner, Honghao Lin, David Woodruff, Vahab Mirrokni for providing the informal proof of the even-case of Knuth’s Cycles problem.

References

- T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner, K. Henningsgard, Y. Kudryashov, A. Meiburg, M. Michelsen, R. Patterson, E. Rodriguez, L. Scharff, V. Shanker, V. Sicca, H. Sowrirajan, A. Swope, M. Tamas, V. Tenev, J. Thomm, H. Williams, and L. Wu. Aristotle: Imo-level automated theorem proving, 2025. URL <https://arxiv.org/abs/2510.01346>.
- M. Carneiro. Metamath zero: The cartesian theorem prover, 2020. URL <https://arxiv.org/abs/1910.10703>.
- J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience, 2025a. URL <https://arxiv.org/abs/2512.17260>.
- L. Chen, J. Gu, L. Huang, W. Huang, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, K. Ma, C. Ren, J. Shen, W. Shi, T. Sun, H. Sun, J. Wang, S. Wang, Z. Wang, C. Wei, S. Wei, Y. Wu, Y. Wu, Y. Xia, H. Xin, F. Yang, H. Ying, H. Yuan, Z. Yuan, T. Zhan, C. Zhang, Y. Zhang, G. Zhang, T. Zhao, J. Zhao, Y. Zhou, and T. H. Zhu. Seed-prover: Deep and broad reasoning for automated theorem proving, 2025b. URL <https://arxiv.org/abs/2507.23726>.
- T. Feng, J. Jung, S. hyun Kim, C. Pagano, S. Gukov, C.-C. Tsai, D. Woodruff, A. Javanmard, A. Mokhtari, D. Hwang, Y. Chervonyi, J. N. Lee, G. Bingham, T. H. Trinh, V. Mirrokni, Q. V. Le, and T. Luong. Aletheia tackles firstproof autonomously, 2026a. URL <https://arxiv.org/abs/2602.21201>.
- T. Feng, T. Trinh, G. Bingham, J. Kang, S. Zhang, S. hyun Kim, K. Barreto, C. Schildkraut, J. Jung, J. Seo, C. Pagano, Y. Chervonyi, D. Hwang, K. Hou, S. Gukov, C.-C. Tsai, H. Choi, Y. Jin, W.-Y. Li, H.-A. Wu, R.-A. Shiu, Y.-S. Shih, Q. V. Le, and T. Luong. Semi-autonomous mathematics discovery with gemini: A case study on the erdős problems, 2026b, 2026b. URL <https://arxiv.org/abs/2601.22401>.
- T. Feng, T. H. Trinh, G. Bingham, D. Hwang, Y. Chervonyi, J. Jung, J. Lee, C. Pagano, S. hyun Kim, F. Pasqualotto, S. Gukov, J. N. Lee, J. Kim, K. Hou, G. Ghiasi, Y. Tay, Y. Li, C. Kuang, Y. Liu, H. Lin, E. Z. Liu, N. Nayakanti, X. Yang, H.-T. Cheng, D. Hassabis, K. Kavukcuoglu, Q. V. Le, and T. Luong. Towards autonomous mathematics research, 2026c. URL <https://arxiv.org/abs/2602.10177>.
- E. First, M. N. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1229–1241, 2023.
- G. Gao, H. Ju, J. Jiang, Z. Qin, and B. Dong. A semantic search engine for mathlib4. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8001–8013, Miami, Florida, USA, Nov. 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.470. URL <https://aclanthology.org/2024.findings-emnlp.470/>.
- C. Greiffenhagen. Checking correctness in mathematical peer review. *Social studies of science*, 54(2): 184–209, 2024.

- D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- T. Hales, M. Adams, G. Bauer, D. T. Dang, J. Harrison, T. L. Hoang, C. Kaliszyk, V. Magron, S. McLaughlin, T. T. Nguyen, T. Q. Nguyen, T. Nipkow, S. Obua, J. Pleso, J. Rute, A. Solovyev, A. H. T. Ta, T. N. Tran, D. T. Trieu, J. Urban, K. K. Vu, and R. Zumkeller. A formal proof of the kepler conjecture. *Forum of mathematics, Pi*, 5, 2017.
- T. C. Hales. A proof of the kepler conjecture. *Annals of mathematics*, 162(3):1065–1185, 2005.
- J. Harrison. Hol light: An overview. In *International Conference on Theorem Proving in Higher Order Logics*, pages 60–66. Springer, 2009.
- Y. Huang and L. F. Yang. Winning gold at IMO 2025 with a model-agnostic verification-and-refinement pipeline. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*, 2025. URL <https://openreview.net/forum?id=svz4xDjRC1>.
- T. Hubert, R. Mehta, L. Sartran, M. Z. Horváth, G. Žužić, E. Wieser, A. Huang, J. Schrittwieser, Y. Schroecker, H. Masoom, O. Bertolli, T. Zahavy, A. Mandhane, J. Yung, I. Beloshapka, B. Ibarz, V. Veeriah, L. Yu, O. Nash, P. Lezeau, S. Mercuri, C. Sönne, B. Mehta, A. Davies, D. Zheng, F. Pedregosa, Y. Li, I. von Glehn, M. Rowland, S. Albanie, A. Velingker, S. Schmitt, E. Lockhart, E. Hughes, H. Michalewski, N. Sonnerat, D. Hassabis, P. Kohli, and D. Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2026. doi: 10.1038/s41586-025-09833-y. URL <https://www.nature.com/articles/s41586-025-09833-y>.
- G. Huet, G. Kahn, and C. Paulin-Mohring. The coq proof assistant a tutorial. *Rapport Technique*, 178: 113, 1997.
- A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- A. Q. Jiang, S. Welleck, J. P. Zhou, T. Lacroix, J. Liu, W. Li, M. Jamnik, G. Lample, and Y. Wu. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=SMA9EAovKMC>.
- G. Lample, T. Lacroix, M.-A. Lachaux, A. Rodriguez, A. Hayat, T. Lavril, G. Ebner, and X. Martinet. Hypertree proof search for neural theorem proving. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 26337–26349. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/a8901c5e85fb8e1823bbf0f755053672-Paper-Conference.pdf.
- H. Lin, Z. Sun, S. Welleck, and Y. Yang. Lean-STar: Learning to interleave thinking and proving. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=S0WZ59UyNc>.
- Y. Lin, S. Tang, B. Lyu, Z. Yang, J.-H. Chung, H. Zhao, L. Jiang, Y. Geng, J. Ge, J. Sun, J. Wu, J. Gesi, X. Lu, D. Acuna, K. Yang, H. Lin, Y. Choi, D. Chen, S. Arora, and C. Jin. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=j4C0nALrgK>.

- P. Lu, R. Gong, S. Jiang, L. Qiu, S. Huang, X. Liang, and S.-C. Zhu. Inter-GPS: Interpretable geometry problem solving with formal language and symbolic reasoning. In C. Zong, F. Xia, W. Li, and R. Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6774–6786, Online, Aug. 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.528. URL <https://aclanthology.org/2021.acl-long.528/>.
- T. Luong and E. Lockhart. Advanced version of gemini with deep think officially achieves gold-medal standard at the international mathematical olympiad. Google DeepMind Blog, July 2025. URL <https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/>. Accessed: 2026-05-02.
- T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. Towards robust mathematical reasoning. In C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 35418–35442, Suzhou, China, Nov. 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1794. URL <https://aclanthology.org/2025.emnlp-main.1794/>.
- T. mathlib Community. The lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, POPL ’20*, page 367–381. ACM, Jan. 2020. doi: 10.1145/3372885.3373824. URL <http://dx.doi.org/10.1145/3372885.3373824>.
- L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pages 625–635. Springer, 2021.
- T. Nipkow, M. Wenzel, and L. C. Paulson. *Isabelle/HOL: a proof assistant for higher-order logic*. Springer, 2002.
- S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- N. Ravi, K. Ying, V. Nesterov, R. Krishnan, E. Uskuplu, B. Xia, J. Aswedige, and L. Nashold. Formalproofbench: Can models write graduate level math proofs that are formally verified? In *ICLR 2026 Workshop: VerifAI-2: The Second Workshop on AI Verification in the Wild*, 2026. URL <https://openreview.net/forum?id=1LdnVndCEG>.
- Z. Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, Z. F. Wu, Z. Gou, S. Ma, H. Tang, Y. Liu, W. Gao, D. Guo, and C. Ruan. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition, 2025. URL <https://arxiv.org/abs/2504.21801>.
- A. K. Taylor, J. Zhang, E. Ji, V. Sahai, H. Deng, Y. Chen, Y. Yuan, D. Wu, J.-C. Gu, K.-W. Chang, N. Peng, A. Sahai, and W. Wang. Taobench: Do automated theorem prover llms generalize beyond mathlib?, 2026. URL <https://arxiv.org/abs/2603.12744>.
- G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 11545–11569. Curran Associates, Inc., 2024. doi: 10.52202/079017-0368. URL https://proceedings.neurips.cc/paper_files

/paper/2024/file/1582eaf9e0cf349e1e5a6ee453100aa1-Paper-Datasets_and_Benchmarks_Track.pdf.

- G. Tsoukalas, A. Kovsharov, S. Shirobokov, A. Surina, M. Firsching, G. Bérczi, F. J. Ruiz, A. Suggala, A. Z. Wagner, E. Wieser, et al. Advancing mathematics research with ai-driven formal proof search. *arXiv preprint arXiv:2605.22763*, 2026.
- S. Varambally, T. Voice, Y. Sun, Z. Chen, R. Yu, and K. Ye. Hilbert: Recursively building formal proofs with informal reasoning. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*, 2025. URL <https://openreview.net/forum?id=ljAHonPrs1>.
- H. Wang, M. Unsal, X. Lin, M. Baksys, J. Liu, M. D. Santos, F. Sung, M. Vinyes, Z. Ying, Z. Zhu, J. Lu, H. de Saxcé, B. Bailey, C. Song, C. Xiao, D. Zhang, E. Zhang, F. Pu, H. Zhu, J. Liu, J. Bayer, J. Michel, L. Yu, L. Dreyfus-Schmidt, L. Tunstall, L. Pagani, M. Machado, P. Bourigault, R. Wang, S. Polu, T. Barroyer, W.-D. Li, Y. Niu, Y. Fleureau, Y. Hu, Z. Yu, Z. Wang, Z. Yang, Z. Liu, and J. Li. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning, 2025. URL <https://arxiv.org/abs/2504.11354>.
- Y. Wu, A. Q. Jiang, W. Li, M. Rabe, C. Staats, M. Jamnik, and C. Szegedy. Autoformalization with large language models. *Advances in neural information processing systems*, 35:32353–32368, 2022.
- H. Xin, Z. Ren, J. Song, Z. Shao, W. Zhao, H. Wang, B. Liu, L. Zhang, X. Lu, Q. Du, W. Gao, H. Zhang, Q. Zhu, D. Yang, Z. Gou, Z. Wu, F. Luo, and C. Ruan. Deepseek-prover-v1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL <https://openreview.net/forum?id=I4YAIwrsXa>.
- R. Xin, C. Xi, J. Yang, F. Chen, H. Wu, X. Xiao, Y. Sun, S. Zheng, and M. Ding. BFS-prover: Scalable best-first tree search for LLM-based automatic theorem proving. In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32588–32599, Vienna, Austria, July 2025b. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1565. URL <https://aclanthology.org/2025.acl-long.1565/>.
- K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 21573–21612. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/4441469427094f8873d0fecb0c4e1cee-Paper-Datasets_and_Benchmarks.pdf.
- K. Zheng, J. M. Han, and S. Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=9ZPegFuFTFv>.

A. Related Work

The intersection of machine learning and formal mathematics has seen significant growth, driven by the development of powerful language models and the availability of formalized benchmarks.

Neural Theorem Proving Early work in neural theorem proving mainly utilize in-house symbolic engines, such as Metamath (Polu and Sutskever, 2020), MMO (Carneiro, 2020) or some dedicated formal language for geometry problems (Lu et al., 2021). Later work such as mathlib (mathlib Community, 2020), LeanDojo (Yang et al., 2023) and MiniF2F (Zheng et al., 2022) pioneered the use of LLMs for generative theorem proving in Lean. They serve as pillar that provide a rich library of known theorems, an interactive environment for step-level search and a descent-level of difficulty evaluation set. To manage the large search space, HyperTree Proof Search (Lample et al., 2022) and related Monte Carlo tree search methods (Lin et al., 2025; Xin et al., 2025b) have been explored. While search-based methods operate at the tactic level, Baldur (First et al., 2023) and DeepSeek-prover-v1.5 (Xin et al., 2025a) explored whole-proof generation, attempting to produce a complete proof in a single step. Another promising direction is guiding formal proof search with informal proofs or sketches. The “draft, sketch, and prove” methodology (Jiang et al., 2023) demonstrated that using an informal proof as a blueprint can significantly guide and improve formal theorem proving. Our work, LEAP, builds on this intuition by utilizing general LLMs to generate informal blueprints and iteratively refine formal proofs based on compiler feedback, but without relying on specialized fine-tuned models for the formalization step.

Specialized Prover Models Recent breakthroughs have often relied on extensive fine-tuning of large models on formal mathematical corpora. Representative works include AlphaProof (Hubert et al., 2026), DeepSeek Prover V2 (Ren et al., 2025), Seed Prover (Chen et al., 2025b), Kimina Prover (Wang et al., 2025) and Goedel Prover V2 (Lin et al., 2026). These models achieve state-of-the-art performance by scaling up training and search on formal systems. However, they require substantial computational resources for training and are highly specialized for specific formal languages. In contrast, LEAP demonstrates that general-purpose LLMs, when placed in a proper agentic environment, can achieve competitive performance without such specialized fine-tuning.

Auto-Formalization Auto-formalization, the task of translating natural language mathematics into formal statements and proofs, is a critical bridge between informal and formal reasoning. Early work relied on neural machine translation techniques (Wu et al., 2022). More recently, LLMs have been used to generate formal statements for training provers at scale, as seen in the auto-formalization pipeline of AlphaProof (Hubert et al., 2026). LEAP utilizes the strong auto-formalization capabilities of general LLMs within its agentic harness to bridge the gap between informal blueprints and formal proofs.

Mathematical Reasoning with LLMs Large Language Models have shown impressive progress in solving natural-language mathematical problems, demonstrating strong capabilities in complex reasoning. Recent advancements, such as OpenAI o1 (Jaech et al., 2024) and DeepSeek R1 (Guo et al., 2025), have demonstrated the effectiveness of scaling reinforcement learning for complex mathematical tasks, achieving high scores on competitive benchmarks like AIME. However, direct evaluation of these models on formal theorem proving benchmarks often yields low solve rates, highlighting the gap between informal reasoning and formal verification. LEAP addresses this by leveraging the strong informal reasoning and instruction-following capabilities of general LLMs within an agentic harness, enabling them to interact with the Lean compiler and iteratively self-correct, thus bridging the formalization gap without specialized fine-tuning.

B. Proof Contexts and Artifacts

This section describes the formal and informal artifacts used by LEAP during proof planning. Formal artifacts correspond to Lean-level objects that are checked by the compiler or represented in the proof DAG, while informal artifacts correspond to natural-language planning objects used to guide direct proving and decomposition.

Formal context. A *proof goal* is a Lean theorem or lemma statement that remains to be proved. The original input theorem is the root proof goal, while lemma statements introduced by decomposition become subgoals in the proof DAG. A *formal proof* is a complete Lean proof of the current proof goal that does not rely on newly proposed unproven lemmas; once accepted by the Lean compiler, the corresponding goal is marked as resolved. A *proof sketch* is a Lean artifact that proves the current goal assuming a set of proposed lemmas. In LEAP, a proof sketch may contain sorry placeholders only for these explicitly proposed lemmas. Thus, a verified proof sketch defines a valid decomposition: once all referenced proposed lemmas are later proved, the current goal is also proved. We present examples of these context using Lean-IMO-Bench Problem 001 and 009 in the Basic Set. **All artifacts, except for the Proof Goal of the root problem, are created by LEAP automatically.**

Example Proof Goal (Lean-IMO-Bench, Basic 001)

```
theorem PBBasic001 : {f :  $\mathbb{Z} \rightarrow \mathbb{Z}$  |  $\forall x y, f (2 * x) + 2 * f y = f (f (x + y))$ }  
  = {0}  $\cup$  {(fun x  $\mapsto$  2 * x + c) | (c :  $\mathbb{Z}$ )} :=  
by sorry
```

Example Formal Proof (Lean-IMO-Bench, Basic 001)

```
import Mathlib

theorem PBBasic001 : {f :  $\mathbb{Z} \rightarrow \mathbb{Z}$  |  $\forall x y, f (2 * x) + 2 * f y = f (f (x + y))$ }  
  = {0}  $\cup$  {(fun x  $\mapsto$  2 * x + c) | (c :  $\mathbb{Z}$ )} :=  
by  
  ext f  
  simp only [Set.mem_setOf_eq, Set.mem_union, Set.mem_singleton_iff, Set.mem_range]  
  constructor  
  · intro h  
    have h1 :  $\forall y, f (f y) = 2 * f y + f 0$  := by  
      -- [Proof details omitted for brevity]  
    have h2 :  $\forall x, f (2 * x) = 2 * f x - f 0$  := by  
      -- [Proof details omitted]  
    have h3 :  $\forall x y, f (x + y) = f x + f y - f 0$  := by  
      -- [Proof details omitted]  
  
    have hc :  $\exists c, c = f 0$  := ⟨f 0, rfl⟩  
    rcases hc with ⟨c, hc_eq⟩  
    have hk :  $\exists k, k = f 1 - c$  := ⟨f 1 - c, rfl⟩  
    rcases hk with ⟨k, hk_eq⟩  
  
    -- ... [Induction steps for h4 omitted] ...  
  
    have h5 :  $\forall x : \mathbb{Z}, f x = k * x + c$  := by  
      -- ... [Negative cases to prove linear form omitted] ...
```

```

have eq_all : ∀ x y : ℤ, k * (2 * x) + c + 2 * (k * y + c) = k * (k * (x + y) +
c) + c := by
  intro x y
  have h_orig := h x y
  simp only [h5] at h_orig
  exact h_orig

have hk_eq : k * (k - 2) = 0 := by
  -- [Algebraic simplification using eq_all 1 0 and eq_all 0 0 omitted]

have hk2 : k = 0 ∨ k = 2 := by
  cases mul_eq_zero.mp hk_eq with
  | inl h1 => left; exact h1
  | inr h2 => right; omega

rcases hk2 with hk0 | hk2
· left
  have eq00 := eq_all 0 0
  rw [hk0] at eq00
  have hc0 : c = 0 := by linarith [eq00]
  ext x
  simp only [Pi.zero_apply]
  have hfx := h5 x
  rw [hk0, hc0] at hfx
  omega
· right
  use c
  ext x
  have hfx := h5 x
  rw [hk2] at hfx
  omega

-- Prove that the derived candidates are indeed solutions mappings
· rintro (rfl | ⟨c, rfl⟩)
  · intro x y
    simp only [Pi.zero_apply, mul_zero, add_zero]
  · intro x y
    dsimp only
    ring

```

Example Proof Sketch (Lean-IMO-Bench, Basic 006)

```

import Mathlib

open Polynomial

-- Shared Definitions from the file environment
def K (c : ℕ → ℤ) : ℤ := (c 1)^2 - (2 : ℤ) * (c 0) * (c 2)

def k_target (c : ℕ → ℤ) : ℕ := max 2 (Int.toNat (K c + (1 : ℤ)))

def esymm_one_target (s : Multiset ℝ) : ℕ → ℝ
| 0 => 0
| k + 1 => s.esymm k

def esymm_two_target (s : Multiset ℝ) : ℕ → ℝ
| 0 => 0

```

```

| 1 => 0
| k + 2 => s.esymm k

-- Supporting Lemmas (with sorry)
lemma root_count_bound_implies_eq (c : ℕ → ℤ) (hc : c 0 ≠ 0) (k : ℕ)
  (h_not_less : ¬ ((∑ i ∈ Finset.Icc 0 k, monomial i (c i)).rootSet ℝ).ncard < k))
  :
  ((∑ i ∈ Finset.Icc 0 k, monomial i (c i)).rootSet ℝ).ncard = k ∧
  (∑ i ∈ Finset.Icc 0 k, monomial i (c i)).natDegree = k := by
  sorry

lemma k_le_K_of_eq (c : ℕ → ℤ) (hc : c 0 ≠ 0) (k : ℕ) (hk : (2 : ℕ) ≤ k)
  (h_eq : ((∑ i ∈ Finset.Icc 0 k, monomial i (c i)).rootSet ℝ).ncard = k)
  (h_deg : (∑ i ∈ Finset.Icc 0 k, monomial i (c i)).natDegree = k) :
  (k : ℤ) ≤ K c := by
  sorry

lemma k_target_ge_two (c : ℕ → ℤ) : (2 : ℕ) ≤ k_target c := by
  sorry

lemma k_target_gt_K (c : ℕ → ℤ) : K c < (k_target c : ℤ) := by
  sorry

-- Main Theorem
theorem PBBasic006 (c : ℕ → ℤ) (hc : c 0 ≠ 0) :
  ∃ k, ((∑ i ∈ Finset.Icc 0 k, monomial i (c i)).rootSet ℝ).ncard < k := by
  use k_target c
  by_contra h
  have h1 := root_count_bound_implies_eq c hc (k_target c) h
  have h2 := k_le_K_of_eq c hc (k_target c) (k_target_ge_two c) h1.1 h1.2
  have h3 := k_target_gt_K c
  linarith

```

Informal context. An *informal proof* is a natural-language plan for proving the current goal directly, without introducing a decomposition into new subgoals. It guides the generation of a complete formal proof. An *informal blueprint* is a higher-level natural-language plan that explains how the current goal can be reduced to useful supporting lemmas. Unlike an informal proof, an informal blueprint may introduce proposed lemmas that are not immediately proved and may include auxiliary lemmas for later proof steps. These proposed lemmas are then translated into formal subgoals and organized in the proof DAG. Examples are shown below:

Example Informal Proof (Lean-IMO-Bench, Basic 001)

Let $S = \{f : \mathbb{Z} \rightarrow \mathbb{Z} \mid \forall x, y \in \mathbb{Z}, f(2x) + 2f(y) = f(f(x+y))\}$ and $T = \{0\} \cup \{x \mapsto 2x + c \mid c \in \mathbb{Z}\}$. We want to show that $S = T$.

Part 1: Proof that $T \subseteq S$

Suppose $f \in T$. Then either f is the constant zero function ($f = 0$) or $f(x) = 2x + c$ for some $c \in \mathbb{Z}$.

- Case 1: If $f = 0$, then for any $x, y \in \mathbb{Z}$, the left-hand side is $f(2x) + 2f(y) = 0 + 2(0) = 0$. The right-hand side is $f(f(x+y)) = f(0) = 0$. The two sides match, so $f \in S$.
- Case 2: If $f(x) = 2x + c$ for some $c \in \mathbb{Z}$, then for any $x, y \in \mathbb{Z}$, the left-hand side is: $f(2x) + 2f(y) = (2(2x) + c) + 2(2y + c) = 4x + 4y + 3c$. The right-hand side is: $f(f(x+y)) = f(2(x+y) + c) = 2(2x + 2y + c) + c = 4x + 4y + 3c$. The two sides are equal, so $f \in S$.

This proves $T \subseteq S$.

Part 2: Proof that $S \subseteq T$

Let $f \in S$. Then for all $x, y \in \mathbb{Z}$, we have:

$$(1) f(2x) + 2f(y) = f(f(x + y)).$$

Substitute $x = 0$ into Eq. (1):

$$f(0) + 2f(y) = f(f(y)).$$

Let $c = f(0)$. Then for all $y \in \mathbb{Z}$, we have:

$$(2) f(f(y)) = 2f(y) + c.$$

Using Eq. (2) on the right-hand side of Eq. (1) (since it holds for any input, including $x + y$), we obtain:

$$(3) f(2x) + 2f(y) = 2f(x + y) + c.$$

Substitute $y = 0$ into Eq. (3):

$$f(2x) + 2f(0) = 2f(x) + c.$$

Since $f(0) = c$, this becomes $f(2x) + 2c = 2f(x) + c$, which yields:

$$(4) f(2x) = 2f(x) - c.$$

Now, substitute Eq. (4) back into the left-hand side of Eq. (3):

$$(2f(x) - c) + 2f(y) = 2f(x + y) + c.$$

Rearranging the terms gives:

$$2f(x + y) = 2f(x) + 2f(y) - 2c.$$

Because the codomain is $\mathbb{Z}$, we can divide both sides by 2 to obtain:

$$(5) f(x + y) = f(x) + f(y) - c.$$

Define a new function $g : \mathbb{Z} \rightarrow \mathbb{Z}$ by $g(x) = f(x) - c$. Then we can rewrite Eq. (5) as:

$$g(x + y) + c = (g(x) + c) + (g(y) + c) - c.$$

Simplifying this yields Cauchy's functional equation:

$$g(x + y) = g(x) + g(y) \text{ for all } x, y \in \mathbb{Z}.$$

Since g satisfies $g(x + y) = g(x) + g(y)$ over integers, it must be a linear function. Let $k = g(1) \in \mathbb{Z}$. By standard induction, we have $g(x) = kx$ for all integers $x \geq 0$ (with $g(0) = 0$). For negative integers, $0 = g(0) = g(x + (-x)) = g(x) + g(-x)$, meaning $g(x) = -g(-x) = -(-kx) = kx$. Thus $g(x) = kx$ for all $x \in \mathbb{Z}$.

As a result, we have $f(x) = kx + c$ for all $x \in \mathbb{Z}$.

To determine the permissible values of k and c , substitute $f(x) = kx + c$ back into the original Eq. (1):

$$\text{Left-hand side: } f(2x) + 2f(y) = k(2x) + c + 2(ky + c) = 2kx + 2ky + 3c.$$

$$\text{Right-hand side: } f(f(x + y)) = k(f(x + y)) + c = k(k(x + y) + c) + c = k^2x + k^2y + (k + 1)c.$$

For the equality $2kx + 2ky + 3c = k^2x + k^2y + (k + 1)c$ to hold for all $x, y \in \mathbb{Z}$, the corresponding coefficients must match.

Setting $x = 0$ and $y = 0$ yields:

$$(6) 3c = (k + 1)c.$$

Setting $x = 1$ and $y = 0$ yields:

$$2k + 3c = k^2 + (k + 1)c.$$

Subtracting Eq. (6) from this gives $2k = k^2$, which simplifies to $k(k - 2) = 0$. Since $k \in \mathbb{Z}$, the only solutions are $k = 0$ or $k = 2$.

- Case A: $k = 0$. Substitute $k = 0$ into Eq. (6): $3c = c \implies 2c = 0 \implies c = 0$. Thus, $f(x) = 0x + 0 = 0$, meaning f is the zero function. Hence $f \in \{0\} \subseteq T$.
- Case B: $k = 2$. Substitute $k = 2$ into Eq. (6): $3c = 3c$, which is true for any $c \in \mathbb{Z}$. Thus, $f(x) = 2x + c$ for some $c \in \mathbb{Z}$. Hence $f \in \{x \mapsto 2x + c \mid c \in \mathbb{Z}\} \subseteq T$.

In all cases, any function $f \in S$ is also in T , proving $S \subseteq T$.

Since both subset inclusions have been established, $S = T$.

Example Informal Blueprint (Lean-IMO-Bench, Basic 006)

High-Level Mathematical Idea

The goal is to prove that $k \leq K(c)$, where $K(c) = c_1^2 - 2c_0c_2$, given that the polynomial $P(x) = \sum_{i=0}^k c_i x^i$ with integer coefficients has degree k and exactly k real roots.

The proof elegantly uses multiset symmetric polynomials and the AM-GM inequality, avoiding rational functions or polynomial derivatives:

1. **Polynomial Roots and Splitting:** Since P has natural degree k and k distinct real roots, it splits completely over $\mathbb{R}$, and the multiset of its roots S has cardinality k .
2. **Vieta's Formulas:** Vieta's formulas express the coefficients c_0, c_1, c_2 in terms of the elementary symmetric polynomials of S :

$$c_0 = c_k(-1)^k E_k$$

$$c_1 = c_k(-1)^{k-1} E_{k-1}$$

$$c_2 = c_k(-1)^{k-2} E_{k-2}$$
 where $E_i = \text{esymm}_i(S)$.
3. **Multiset Identity:** We construct a new multiset Y by mapping each root $x \in S$ to the product of all other roots, $(S \setminus \{x\})\text{.prod}$. Purely multiset inductive identities show:

$$\sum Y = E_{k-1}$$

$$\text{esymm}_2(Y) = E_k E_{k-2}$$

$$\prod Y = (E_k)^{k-1}$$
4. **Sum of Squares:** For the multiset $Z = c_k Y$, we evaluate the sum of its squares $W = \{z^2 \mid z \in Z\}$. By the relation $(\sum Z)^2 = \sum(Z^2) + 2\text{esymm}_2(Z)$, we can algebraically compute $\sum W = c_1^2 - 2c_0c_2 = K(c)$.
5. **Integer Product Bound:** The product of Z evaluates to $c_k((-1)^k c_0)^{k-1}$. Since P has degree k and $c_0 \neq 0$, both c_0 and c_k are non-zero integers. Thus, the product of Z is a non-zero integer, implying $\prod W = (\prod Z)^2 \geq 1$.
6. **AM-GM Inequality:** Applying the AM-GM inequality to the multiset W (which consists of k non-negative real numbers whose product is ≥ 1), we obtain $\sum W \geq k \implies K(c) \geq k$.

Required Global Definitions, Variables, or Structures

No new definitions, axioms, or structures are needed. We use purely standard Mathlib components (like `Multiset`, `Polynomial`, and `esymm`).

Smaller Lemmas (Subproblems)

```
lemma coeff_of_sum_Icc (c : ℕ → ℤ) (k : ℕ) (hk : (2 : ℕ) ≤ k)
  (i : ℕ) (hi : i ≤ k) :
  (∑ j ∈ Finset.Icc 0 k, Polynomial.monomial j (c j)).coeff i =
  c i
```

Purpose: Simplifies polynomial coefficient extraction from the given sum format.

```
lemma card_roots_eq_of_ncard_rootSet {k : ℕ} {P : Polynomial ℝ}
  (h_deg : P.natDegree = k)
  (h_ncard : (P.rootSet ℝ).ncard = k) :
  P.roots.card = k
```

Purpose: Shows that a polynomial with k distinct real roots and degree k has exactly k roots counting multiplicities.

```
lemma multiset_map_erase_prod_sum {R : Type*} [CommRing R] (s :
  Multiset R) :
  (s.map (fun x => (s.erase x).prod)).sum = s.esymm (s.card - 1)
```

Purpose: Multiset identity relating the sum of element-wise excluded products to E_{k-1} .

```
lemma multiset_map_erase_prod_esymm_two {R : Type*} [CommRing R]
  (s : Multiset R) :
  (s.map (fun x => (s.erase x).prod)).esymm 2 = s.prod * s.esymm
  (s.card - 2)
```

Purpose: Multiset identity relating the 2nd elementary symmetric polynomial of excluded products to $E_k E_{k-2}$.

```
lemma multiset_sum_sq_eq {R : Type*} [CommRing R] (s : Multiset R)
  :
  (s.map (fun x => x^2)).sum = (s.sum)^2 - (2 : R) * s.esymm 2
```

Purpose: Expresses the sum of squares of a multiset in terms of its sum and its 2nd elementary symmetric polynomial.

```
lemma multiset_map_erase_prod_prod {R : Type*} [CommRing R] (s :
Multiset R) :
  (s.map (fun x => (s.erase x).prod)).prod = s.prod ^ (s.card -
1)
```

Purpose: Computes the full product of the excluded products multiset.

```
lemma multiset_sum_ge_card_of_prod_ge_one (W : Multiset R) (hw :
  ∀ x ∈ W, 0 ≤ x) (hp : (1 : R) ≤ W.prod) :
  (W.card : R) ≤ W.sum
```

Purpose: The AM-GM inequality specialized for a multiset whose product is at least 1, proving that the sum is bounded below by its cardinality.

Proof Body Outline

1. Define P as the sum $\sum_{i \in \text{Finset.Icc } 0 \ k} \text{monomial } i(c_i)$ and P_R as $P.\text{map } (\text{algebraMap } \mathbb{Z} \ \mathbb{R})$.
2. Apply `coeff_of_sum_Icc` to assert $P_R.\text{coeff } i = (c_i : \mathbb{R})$ for $i \in \{0, 1, 2, k\}$.
3. Establish $P_R.\text{roots}.card = k$ using `card_roots_eq_of_ncard_rootSet` and the natural degree injectivity.
4. Establish that $P_R.\text{splits } (\text{RingHom.id } \mathbb{R})$ follows from `Polynomial.splits_iff_card_roots`.
5. Let $s = P_R.\text{roots}$. Invoke Vieta's formulas (`Polynomial.coeff_eq_esymm_roots_of_splits`) to express c_0, c_1 , and c_2 in terms of $s.\text{esymm } i$.
6. Define multisets Y and Z matching the theoretical blueprint. Use `multiset_sum_sq_eq`, `multiset_map_erase_prod_sum`, and `multiset_map_erase_prod_esymm_two` to show that the sum of the squared elements of Z expands algebraically to exactly $(c_1^2 - 2c_0c_2 : \mathbb{R}) = (K(c) : \mathbb{R})$.
7. Use `multiset_map_erase_prod_prod` to find $Z.\text{prod} = c_k((-1)^k c_0)^{k-1}$.
8. Observe that since c_0 and c_k are non-zero integers, their algebraic combination $Z.\text{prod}$ represents a non-zero integer, so its square (the product of $W = Z^2$) is ≥ 1 .
9. Feed W to `multiset_sum_ge_card_of_prod_ge_one` to deduce that $W.\text{sum} \geq W.\text{card}$.
10. Using $W.\text{card} = k$ and $W.\text{sum} = (K(c) : \mathbb{R})$, deduce $(k : \mathbb{R}) \leq (K(c) : \mathbb{R})$. Use `norm_cast` to translate this back to $(k : \mathbb{Z}) \leq K(c)$.